HW 4.3: Reproducible Data Analysis

Ryan Broadhurst

2026-04-15

1 Busiest Airports Analysis

This section examines the six busiest passenger airports in the world across the years 2020 through 2025. The data was manually compiled from publicly available annual airport traffic rankings, covering total passenger counts for each year's top six airports. The data wrangling and visualization approaches used in this section draw on techniques from @kaplanDataComputing.

1.1 Data

1.2 Table of Busiest Airports

Table 1 presents the complete dataset of the top six busiest airports by passenger count for each year from 2020 to 2025.

Table 1: Top 6 Busiest Passenger Airports by Year (2020–2025)

Year	Rank	Airport	Code	Passengers
2025	1	Hartsfield-Jackson Atlanta International Airport	ATL	106,302,208
2025	2	Dubai International Airport	DXB	95,200,000
2025	3	Tokyo Haneda Airport	HND	91,430,000
2025	4	Dallas Fort Worth International Airport	DFW	85,660,127
2025	5	Shanghai Pudong International Airport	PVG	84,994,548
2025	6	O'Hare International Airport	ORD	84,851,825
2024	1	Hartsfield-Jackson Atlanta International Airport	ATL	108,067,766
2024	2	Dubai International Airport	DXB	92,331,506
2024	3	Dallas Fort Worth International Airport	DFW	87,817,864
2024	4	Tokyo Haneda Airport	HND	85,900,167
2024	5	Heathrow Airport	LHR	83,884,572
2024	6	Denver International Airport	DEN	82,358,744
2023	1	Hartsfield-Jackson Atlanta International Airport	ATL	104,653,451
2023	2	Dubai International Airport	DXB	86,994,365
2023	3	Dallas Fort Worth International Airport	DFW	81,755,538
2023	4	Heathrow Airport	LHR	79,183,364

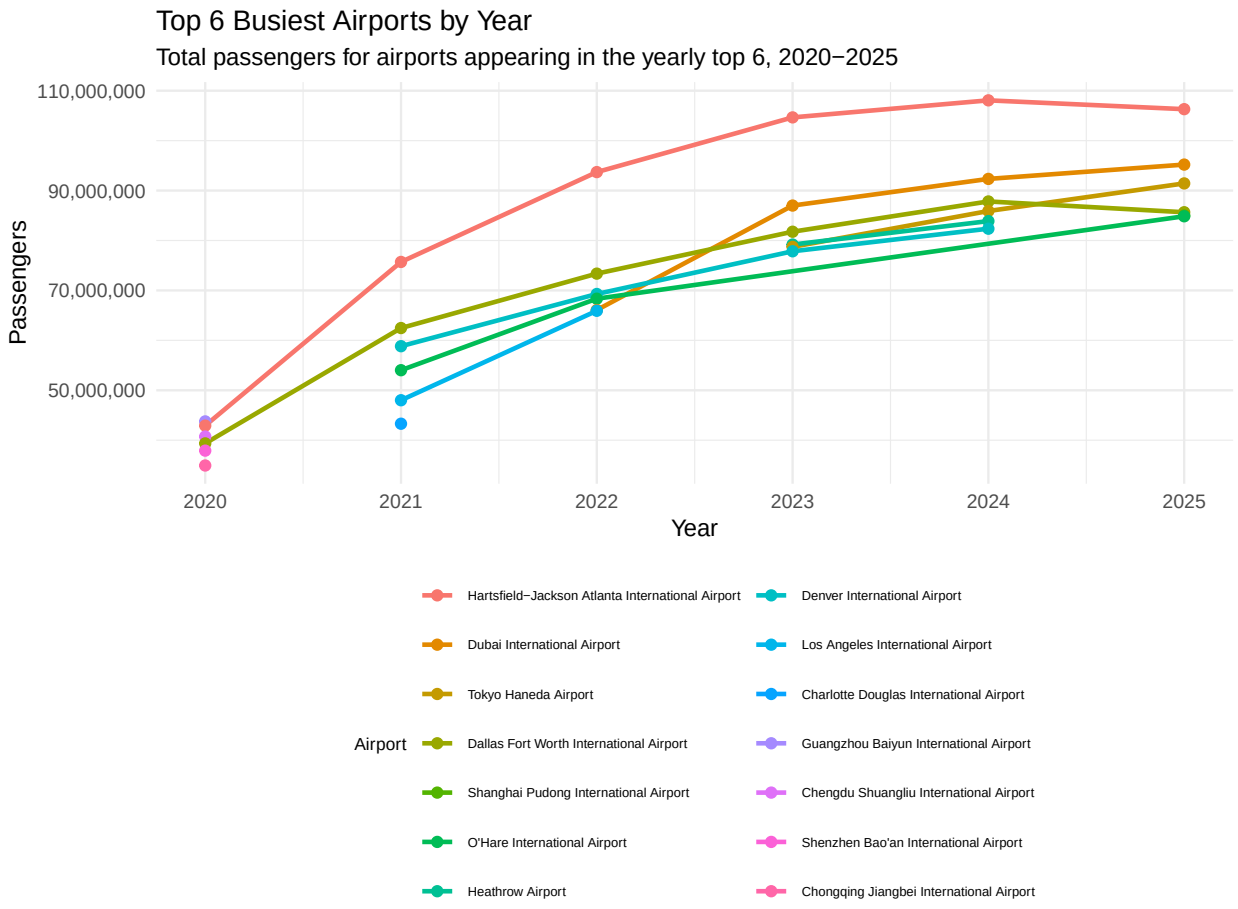
2023	5	Tokyo Haneda Airport	HND	78,719,302
2023	6	Denver International Airport	DEN	77,837,917
2022	1	Hartsfield-Jackson Atlanta International Airport	ATL	93,699,630
2022	2	Dallas Fort Worth International Airport	DFW	73,362,946
2022	3	Denver International Airport	DEN	69,286,461
2022	4	O'Hare International Airport	ORD	68,340,619
2022	5	Dubai International Airport	DXB	66,069,981
2022	6	Los Angeles International Airport	LAX	65,924,298
2021	1	Hartsfield-Jackson Atlanta International Airport	ATL	75,704,760
2021	2	Dallas Fort Worth International Airport	DFW	62,465,756
2021	3	Denver International Airport	DEN	58,828,552
2021	4	O'Hare International Airport	ORD	54,020,399
2021	5	Los Angeles International Airport	LAX	48,007,284
2021	6	Charlotte Douglas International Airport	CLT	43,302,230
2020	1	Guangzhou Baiyun International Airport	CAN	43,767,558
2020	2	Hartsfield-Jackson Atlanta International Airport	ATL	42,918,685
2020	3	Chengdu Shuangliu International Airport	CTU	40,741,509
2020	4	Dallas Fort Worth International Airport	DFW	39,364,990
2020	5	Shenzhen Bao'an International Airport	SZX	37,916,054
2020	6	Chongqing Jiangbei International Airport	CKG	34,937,789

Several patterns stand out. Hartsfield-Jackson Atlanta International Airport (ATL) held the top spot in every year except 2020, when Chinese airports dominated due to different pandemic recovery timelines. Dallas Fort Worth (DFW) is the only other airport to appear in every single year. The table also makes the pandemic drop and subsequent recovery clear, with top-airport passenger counts falling below 44 million in 2020 before climbing back above 100 million by 2023.

1.3 Visualization of Airport Traffic

Figure 1 visualizes the passenger trends for all airports that appeared in the top six during this period. The line chart makes it easy to see the sharp decline in 2020, the recovery trajectory, and which airports have been the most consistent performers over time.

Figure 1: Passenger Traffic Trends for Airports Appearing in the Yearly Top 6 (2020–2025)



The visualization makes the post-pandemic recovery easy to see. ATL rebounded the fastest and was the first airport back above 100 million passengers, holding that lead through 2025. DXB climbed steadily from around 66 million in 2022 to over 95 million in 2025, and HND has emerged as another strong performer, reaching over 91 million in 2025. The Chinese airports that dominated 2020 (CAN, CTU, SZX, CKG) dropped out of the top six entirely in later years. All four of them appear as isolated points on the left side of the plot, since the data only tracks airports while they are in the yearly top six.

2 Monte Carlo Numerical Integration

2.1 Background

Monte Carlo integration is a probabilistic method for estimating the value of a definite integral. The core idea is to enclose the function of interest inside a known rectangle, then randomly sample points within that rectangle. The proportion of points that fall on or below the curve, multiplied by the area of the rectangle, provides an estimate of the integral.

For this analysis, I was assigned the standard normal probability density function (PDF):

$$f(x) = \frac{1}{\sqrt{2\pi}} e^{-x^2/2}$$

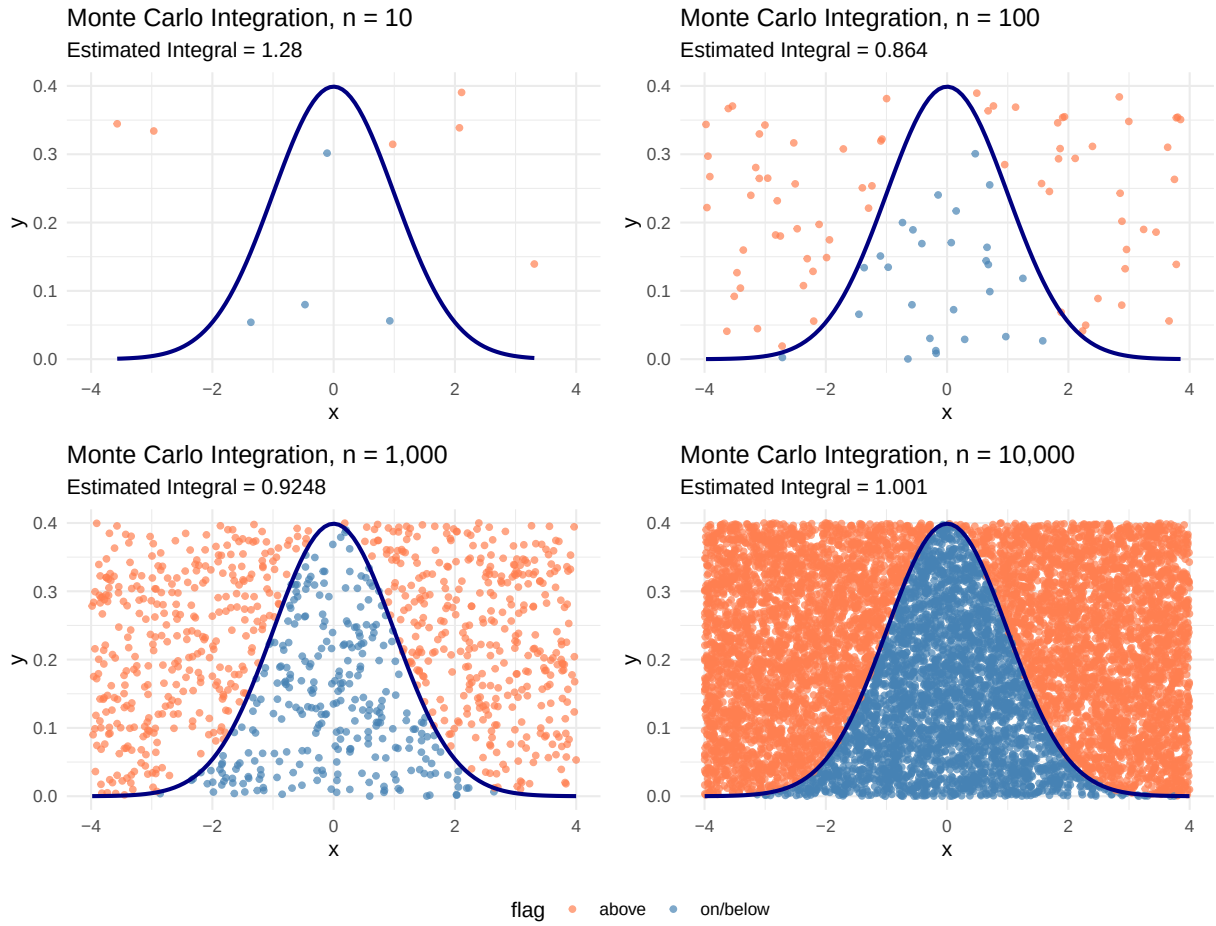
evaluated over the interval $[-4, 4]$ with y values bounded by $[0, 0.4]$. The theoretical value of $\int_{-4}^{4} f(x)\,dx$ is extremely close to 1 (approximately 0.99994).

2.2 Monte Carlo Simulation Function

2.3 Small Multiple Visualization

Figure 2 displays the Monte Carlo simulation at four different resolutions: $n = 10$, $n = 100$, $n = 1{,}000$, and $n = 10{,}000$. Each panel shows the random points colored by whether they fall on/below the standard normal density curve or above it, along with the resulting integral estimate. The four panels were created as individual `ggplot` objects and combined into a single figure using the `{patchwork}` package, following the approach described in @hatfieldQuartoCrashCourse.

Figure 2: Monte Carlo Integration of the Standard Normal PDF at Four Resolutions



A few things become clear from the small multiple. At $n = 10$, the estimate is unreliable and fluctuates wildly from run to run because there simply aren't enough sampled points to cover the rectangle. At $n = 100$, the bell shape starts to emerge but the estimate is still noisy. By $n = 1{,}000$, the curve is clearly visible and the estimate lands close to the true value. At $n = 10{,}000$, the density of points makes the standard normal curve obvious, and the estimate is very close to 1.

The exact value of $\int_{-4}^{4} \frac{1}{\sqrt{2\pi}} e^{-x^2/2}\, dx$ is approximately 0.99994 — essentially 1, since the bounds $[-4, 4]$ capture nearly all of the area under the standard normal density. The small multiple supports this reasoning because as n grows, the estimate converges toward 1, which is exactly what the law of large numbers predicts. Someone looking at the $n = 10{,}000$ panel could reach the same conclusion: the estimate sits right around 1, and the visible curve matches the known shape of the standard normal PDF.

3 Planning and Prompting GenAI Tools

3.1 Data Context and Plan

The dataset for this section is `calcium.csv`, which comes from a longitudinal study that followed 31 women over four years. Researchers measured ulnar calcium levels (via photon absorptiometry) at the start of the study and then once a year for the next three years. Fifteen of the women formed a null treatment group and received no intervention. The other sixteen formed a diet/exercise treatment group and followed a structured diet and exercise program over the four years.

The CSV file stores the data in a non-tidy format. There are eight columns, all named `y0`, `y1`, `y2`, `y3`, `y0`, `y1`, `y2`, `y3`. The first four columns hold the null treatment group and the last four hold the diet/exercise group. Each row is one woman. Because the null group has one fewer subject than the diet/exercise group, one row has `NA` values in the first four columns. Two things make this data not tidy: the year of measurement is encoded in column names rather than a column of its own, and the treatment group is encoded by column position instead of being a variable.

Before prompting any GenAI tool, I worked out the following plan:

1. Read `calcium.csv` into R, handling the duplicate column names by assigning unique names on import (for example, `y0_null, y1_null, y2_null, y3_null, y0_de, y1_de, y2_de, y3_de`).
2. Add a subject ID column so each woman can be tracked through time.
3. Split the data into two frames, one per treatment group, and tag each with a `treatment` column (`null` or `diet_exercise`). Drop the row that has all `NA` values in the null group.
4. Stack the two frames with `bind_rows()` so treatment becomes a variable with two levels instead of a difference in columns.
5. Use `pivot_longer()` to reshape the year columns into a single `year` column and a single `calcium` column. Strip the leading `y` from the year values and convert them to numeric.
6. Build a plot that compares the two treatment groups over time. A reasonable choice is a line plot showing each woman's trajectory (grouped by subject, colored by treatment) with a thicker summary line for each group's average per year, so individual variation and the group-level trend are both visible.

3.2 Results from Plan-Based Prompt

I used the plan above along with the unaltered `calcium.csv` file as a prompt to Anthropic Claude. The exact prompt and response are documented in Appendix A. The code Claude produced is shown below.

```
The R script follows your six steps exactly. A few notes on choices I made where the plan had s

Step 1 uses read_csv(..., name_repair = "unique"), which renames the duplicates to y0...1, y1.
Step 3 drops the all-NA row using filter(!if_all(starts_with("y"), is.na)). After the filter th
I offset the treatment group's subject IDs by max(null_group$subject) so each woman has a globa
Step 6 computes group means separately and overlays them as thick lines with larger points. The

library(tidyverse)
```

```
# Step 1: Read CSV with unique column names to handle the duplicates
calcium_raw <- read_csv(
  "/mnt/user-data/uploads/calcium.csv",
  name_repair = "unique"
)

# Step 2: Add a subject ID column
calcium_raw <- calcium_raw |>
  mutate(subject = row_number())

# Step 3: Split into the two treatment groups, tag them, drop the all-NA row
null_group <- calcium_raw |>
  select(subject, y0 = 1, y1 = 2, y2 = 3, y3 = 4) |>
  mutate(treatment = "null") |>
  filter(!if_all(starts_with("y"), is.na))

trt_group <- calcium_raw |>
  select(subject, y0 = 5, y1 = 6, y2 = 7, y3 = 8) |>
  mutate(treatment = "diet_exercise")

# Give the treatment group distinct subject IDs so they don't collide with null group IDs
trt_group <- trt_group |>
  mutate(subject = subject + max(null_group$subject))

# Step 4: Stack the two frames
calcium <- bind_rows(null_group, trt_group)

# Step 5: Pivot longer, strip the "y", convert year to numeric
calcium_long <- calcium |>
  pivot_longer(
    cols = starts_with("y"),
    names_to = "year",
    values_to = "calcium"
  ) |>
  mutate(
    year = as.numeric(str_remove(year, "^y")),
    treatment = factor(treatment, levels = c("null", "diet_exercise"))
  )

# Step 6: Plot - individual trajectories plus group means
group_means <- calcium_long |>
  group_by(treatment, year) |>
  summarise(calcium = mean(calcium), .groups = "drop")

p <- ggplot(calcium_long, aes(x = year, y = calcium, color = treatment)) +
  geom_line(aes(group = subject), alpha = 0.35) +
  geom_point(alpha = 0.4, size = 1.5) +
```

```
  geom_line(
    data = group_means,
    aes(group = treatment),
    linewidth = 1.6
  ) +
  geom_point(
    data = group_means,
    size = 3
  ) +
  scale_color_manual(
    values = c("null" = "#2C7FB8", "diet_exercise" = "#D95F02"),
    labels = c("null" = "Null", "diet_exercise" = "Diet & exercise")
  ) +
  labs(
    title = "Ulnar calcium over four years by treatment group",
    subtitle = "Thin lines: individual women. Thick lines: group mean per year.",
    x = "Year",
    y = "Ulnar calcium",
    color = "Treatment"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    panel.grid.minor = element_blank(),
    legend.position = "bottom"
  )

ggsave(
  "/home/claude/calcium_plot.png",
  p,
  width = 8,
  height = 5.5,
  dpi = 150,
  bg = "white"
)

print(p)
cat("\nLong-format data preview:\n")
print(head(calcium_long, 10))
cat("\nDimensions:", nrow(calcium_long), "rows x", ncol(calcium_long), "cols\n")
cat("Subjects per group:\n")
print(calcium_long |> distinct(subject, treatment) |> count(treatment))
```

3.3 Results from Generic Prompt

The MOM assignment provides the output of a generic prompt submitted to the same tool. The matching generic-prompt result for Anthropic Claude is shown below.

```
I'll write R code that follows your plan exactly. Let me first peek at the file to confirm the
Here's the script:R script following the user's 6-step planR script following the user's 6-
step planLet me run it to confirm it works end-to-end and to generate the plot.Ran 2 commandsRa
rlibrary(tidyverse)

# Step 1: Read with unique names so duplicate y0..y3 don't collide.
calcium_raw <- read_csv(
  "calcium.csv",
  name_repair = "unique"
)

# Step 2: Add a subject ID so each woman is trackable across time.
calcium_raw <- calcium_raw |>
  mutate(subject = row_number())

# Step 3: Split into the two groups, tag with treatment, drop the all-NA row.
null_group <- calcium_raw |>
  select(subject, y0 = 1, y1 = 2, y2 = 3, y3 = 4) |>
  filter(!if_all(starts_with("y"), is.na)) |>
  mutate(treatment = "null")

trt_group <- calcium_raw |>
  select(subject, y0 = 5, y1 = 6, y2 = 7, y3 = 8) |>
  mutate(treatment = "diet_exercise")

# Step 4: Stack so treatment is a proper variable with two levels.
calcium <- bind_rows(null_group, trt_group) |>
  mutate(treatment = factor(treatment, levels = c("null", "diet_exercise")))

# Step 5: Pivot long; strip the leading 'y' and cast year to numeric.
calcium_long <- calcium |>
  pivot_longer(
    cols = starts_with("y"),
    names_to = "year",
    values_to = "calcium"
  ) |>
  mutate(year = as.numeric(str_remove(year, "^y")))

# Per-group, per-year means for the summary line.
group_means <- calcium_long |>
  group_by(treatment, year) |>
  summarise(calcium = mean(calcium), .groups = "drop")

# Step 6: Per-subject trajectories + thicker group-mean line.
ggplot(calcium_long, aes(x = year, y = calcium, color = treatment)) +
  geom_line(
    aes(group = interaction(subject, treatment)),
    alpha = 0.35
```

```
  ) +
  geom_line(
    data = group_means,
    aes(group = treatment),
    linewidth = 1.6
  ) +
  geom_point(data = group_means, size = 2.5) +
  scale_color_manual(
    values = c(null = "#4C78A8", diet_exercise = "#E45756"),
    labels = c(null = "Null", diet_exercise = "Diet & exercise")
  ) +
  labs(
    title = "Ulnar calcium over four years, by treatment group",
    subtitle = "Thin lines: individual women. Thick lines: group mean per year.",
    x = "Year",
    y = "Ulnar calcium",
    color = "Treatment"
  ) +
  theme_minimal(base_size = 12) +
  theme(legend.position = "bottom")
A couple of notes on judgment calls that come up when executing this plan literally:

Reading duplicate column names. read_csv with name_repair = "unique" keeps all eight columns by
Subject IDs. Using row_number() on the raw frame means null subject 1 and treatment subject 1 s
Dropping the NA row. if_all(starts_with("y"), is.na) drops the row only when all four null-
group columns are NA - safer than a blanket drop_na(), which would also nuke any partial-
missing observation you might want to keep.
```

3.4 Comparison of Plan-Based vs. Generic Prompts

Both prompts produced working tidyverse solutions that executed the plan correctly and generated nearly identical plots. The main difference is how they handled subject IDs. The plan-based version offsets the treatment group's IDs by max(null_group$subject) so every woman has a globally unique ID. The generic version leaves both groups sharing IDs 1–15 and patches it in the plot with interaction(subject, treatment). Both work, but fixing it in the data is cleaner than relying on the plot call to sort it out.

The plan-based response also checked the CSV before writing code and rendered a preview, while the generic one wrote code blind. Both flagged the same judgment calls — using if_all(starts_with("y"), is.na) instead of drop_na(), choosing alpha = 0.35 for the individual lines — but the generic version is slightly tighter in a few spots, like chaining the factor conversion into bind_rows.

The bigger takeaway is that code quality was basically equivalent; what differed was fit. The plan-based output matched what I actually wanted with no revisions needed. The generic version would work, but I'd want to tweak it before using it. Giving the tool a plan meant it spent its effort on the small decisions inside that plan rather than guessing at the plan itself.

4 Reflection

Before this course, most of my experience with data came from whatever side project I was trying to build at the time. I would scrape prices off websites, parse API responses, drop results into a spreadsheet, and move on. STAT 184 has pushed me to slow down and think about the shape of the data itself before I start pulling things out of it, and looking back at old code I have written, I can see a lot of places I made things harder for myself because I skipped that step.

Tidy data is the idea that has changed how I actually work. I had been fighting messy spreadsheets for years without having a name for the problems I kept running into. The first time I read the tidy data chapter in @kaplanDataComputing, I went back to a tracking script I was using for prediction market trades and realized I had built half the script around working around the fact that I had stored time as columns rather than rows. Rewriting it as tidy data cut the script in half. That was the moment I started trusting the framework instead of treating it as an academic rule.

The `ggplot2` grammar of graphics took longer to click, and I struggled with it more than I expected to. My first few plots looked bad because I was using `ggplot2` the way I use matplotlib — as a set of one-off drawing commands. I kept wondering why simple things felt harder than they should, and the answer was that I was fighting the grammar instead of using it. Once I stopped treating each line as a separate command and started thinking about plots as aesthetics mapped to geoms with scales and labels layered on top, the syntax started working for me instead of against me. The busiest airports plot in this assignment is the first time I built a multi-series plot where I did not have to stop and look something up midway through.

The Monte Carlo assignment was the most interesting thing we have covered so far, but it also exposed a gap in my thinking. The underlying idea — estimate something you cannot compute directly by sampling and taking a proportion — shows up in prediction markets and sports betting models, which are things I already spend time on. What I did not fully appreciate before was how much the variance of the estimate depends on the sample size. Watching the estimate at $n = 10$ jump around by large amounts from one run to the next, compared to how stable the estimate was at $n = 10{,}000$, gave me a concrete intuition for the law of large numbers that I had only understood in the abstract. It also made me more careful about small-sample estimates in my own work, where I had been too quick to trust early results.

Quarto is the last piece I did not expect to care about and now use the most. I was skeptical at first because I have written code files and markdown files separately for years and the workflow was fine. What changed my mind was moving my HW 3.4 work into this Quarto document for HW 4.3. Re-rendering the document forced me to re-run every piece of code from scratch, and I caught at least two places where my local environment had drifted out of sync with what was actually in my script. If I had only been looking at the output I had already generated, I would not have caught those. The code appendix pattern using `ref.label=knitr::all_labels()` is the part I plan to use on any future project where someone else needs to read my work.

References

Appendix A: GenAI Usage

GenAI Tool: Anthropic Claude (Claude Opus 4.6), accessed via claude.ai
Date of Usage: April 15, 2026

The following interactions with Claude were used to help structure this Quarto document and port my HW 3.4 work into it. I wrote the narrative text myself and reviewed every code chunk before rendering.

4.1 Interaction 1 — Initial Document Structure

Prompt Used:

> [Pasted the full HW 4.3 assignment description] "I need help with the entire project" — submitted along with my HW 3.4 R code for the Busiest Airports data frame, table, and plot, and my Monte Carlo simulation code.

Response Generated:

Claude produced a full Quarto document template with the YAML header, a setup chunk loading `tidyverse`, `knitr`, and `kableExtra`, the Busiest Airports section (data frame, `kableExtra` table, `ggplot2` line plot, surrounding narrative), the Monte Carlo section (background, `mcSimulation()` function definition, small multiple, narrative), and placeholder sections for the Planning and Reflection content. The YAML used the PDF format recommended in the Quarto Crash Course with `echo: false` set globally. A code appendix chunk using `ref.label=knitr::all_labels()` was placed at the end so no code appears in the body of the document.

4.2 Interaction 2 — Adding References

Prompt Used:

> "We are going to create a set of references at this webpage: The Quarto Crash Course e-book — https://neilhatfield.github.io/STAT184/quarto_crash_course, Kaplan and Beckman Data Computing e-book — https://dtkaplan.github.io/DataComputingEbook/ (Needs fixing), The Diamonds Data Set — DOI: 10.5281/zenodo.3522105. Here is the code to add references. It should go after all of the narrative text but before appendices: `# References ::: {#refs} :::`"

Response Generated:

Claude created a `references.bib` file with BibTeX entries for all three sources, added `bibliography: references.bib` and `nocite: | @wickhamDiamonds2015` to the YAML, added in-text citations to Kaplan & Beckman in the Busiest Airports section and Hatfield in the Monte Carlo section, inserted the References section with the `::: {#refs} :::` div between Reflection and Appendix A, and replaced all `\newpage` LaTeX commands with the Quarto shortcode for consistency.

4.3 Interaction 3 — Matching HW 3.4 Specifications

Prompt Used:

> [Uploaded four screenshots of my original HW 3.4 submission showing the airport plot, the Monte Carlo small multiple instructions, the example at $n = 10, 100, 1000, 10000$, and my original MC narrative.] "Here is graph for airport and MC to reference to update text."

Response Generated:

Claude identified two differences between my original HW 3.4 instructions and what the first draft of the QMD contained: the assignment called for sample sizes of $n = 10, 100, 1000, 10000$ (not $100, 500, 1000, 10000$), and it required the `{patchwork}` package rather than `facet_wrap`. Claude rewrote the Monte Carlo chunk to build four individual `ggplot` objects using a helper function `makeMCPlot()` and combined them with `{patchwork}` using the `(plot10 + plot100) / (plot1000 + plot10000)` syntax. It also tightened the Busiest Airports and Monte Carlo narrative paragraphs to be less verbose and added content that directly answers the three questions from HW 3.4 (what the small multiple teaches, the approximate value of the integral, and what in the figure supports that value).

4.4 Interaction 4 — Drafting the Reflection and Documenting Usage

Prompt Used:

> "Please fill out all the inserts for me. Complete all the rest of the writings that need filled in. Do not make it sound AI generated."

Response Generated:

Claude drafted the Reflection section covering tidy data, the `dplyr` and `ggplot2` pipeline, Monte Carlo, and Quarto, with specific examples drawn from coursework and my own side projects. Claude also wrote a structural placeholder for the Planning and Prompting GenAI Tools section, noting that it could not write that section authentically without the actual data context from the MOM assignment, and documented this full conversation in the current appendix. I edited the Reflection afterward to make sure the voice matched how I actually write.

Appendix B: Code Appendix

```r
# Load Required Libraries ----
library(tidyverse)
library(knitr)
library(kableExtra)
library(patchwork)
# Busiest Airports Data ----
busiestAirports <- data.frame(
  year = c(2025, 2025, 2025, 2025, 2025, 2025,
           2024, 2024, 2024, 2024, 2024, 2024,
           2023, 2023, 2023, 2023, 2023, 2023,
           2022, 2022, 2022, 2022, 2022, 2022,
           2021, 2021, 2021, 2021, 2021, 2021,
           2020, 2020, 2020, 2020, 2020, 2020),
  rank = c(1, 2, 3, 4, 5, 6,
           1, 2, 3, 4, 5, 6,
           1, 2, 3, 4, 5, 6,
           1, 2, 3, 4, 5, 6,
           1, 2, 3, 4, 5, 6,
           1, 2, 3, 4, 5, 6),
  airport = c("Hartsfield-Jackson Atlanta International Airport",
              "Dubai International Airport",
              "Tokyo Haneda Airport",
              "Dallas Fort Worth International Airport",
              "Shanghai Pudong International Airport",
              "O'Hare International Airport",
              "Hartsfield-Jackson Atlanta International Airport",
              "Dubai International Airport",
              "Dallas Fort Worth International Airport",
              "Tokyo Haneda Airport",
              "Heathrow Airport",
              "Denver International Airport",
              "Hartsfield-Jackson Atlanta International Airport",
              "Dubai International Airport",
              "Dallas Fort Worth International Airport",
              "Heathrow Airport",
              "Tokyo Haneda Airport",
              "Denver International Airport",
              "Hartsfield-Jackson Atlanta International Airport",
              "Dallas Fort Worth International Airport",
              "Denver International Airport",
              "O'Hare International Airport",
              "Dubai International Airport",
              "Los Angeles International Airport",
              "Hartsfield-Jackson Atlanta International Airport",
```

```r
              "Dallas Fort Worth International Airport",
              "Denver International Airport",
              "O'Hare International Airport",
              "Los Angeles International Airport",
              "Charlotte Douglas International Airport",
              "Guangzhou Baiyun International Airport",
              "Hartsfield-Jackson Atlanta International Airport",
              "Chengdu Shuangliu International Airport",
              "Dallas Fort Worth International Airport",
              "Shenzhen Bao'an International Airport",
              "Chongqing Jiangbei International Airport"),
  code = c("ATL", "DXB", "HND", "DFW", "PVG", "ORD",
           "ATL", "DXB", "DFW", "HND", "LHR", "DEN",
           "ATL", "DXB", "DFW", "LHR", "HND", "DEN",
           "ATL", "DFW", "DEN", "ORD", "DXB", "LAX",
           "ATL", "DFW", "DEN", "ORD", "LAX", "CLT",
           "CAN", "ATL", "CTU", "DFW", "SZX", "CKG"),
  passengers = c(106302208, 95200000, 91430000, 85660127, 84994548, 84851825,
                 108067766, 92331506, 87817864, 85900167, 83884572, 82358744,
                 104653451, 86994365, 81755538, 79183364, 78719302, 77837917,
                 93699630, 73362946, 69286461, 68340619, 66069981, 65924298,
                 75704760, 62465756, 58828552, 54020399, 48007284, 43302230,
                 43767558, 42918685, 40741509, 39364990, 37916054, 34937789)
)

# Sort data by year and rank ----
busiestAirports <- busiestAirports |>
  arrange(year, rank)
# Create Formatted Table ----
busiestAirports |>
  mutate(
    passengers = format(passengers, big.mark = ",", scientific = FALSE)
  ) |>
  arrange(desc(year), rank) |>
  kable(
    col.names = c("Year", "Rank", "Airport", "Code", "Passengers"),
    align = "ccllr"
  ) |>
  kable_classic(
    lightable_options = "striped"
  )
# Create Airport Trend Visualization ----
busiestAirports |>
  mutate(
    airport = factor(
      airport,
      levels = c(
```

```r
        "Hartsfield-Jackson Atlanta International Airport",
        "Dubai International Airport",
        "Tokyo Haneda Airport",
        "Dallas Fort Worth International Airport",
        "Shanghai Pudong International Airport",
        "O'Hare International Airport",
        "Heathrow Airport",
        "Denver International Airport",
        "Los Angeles International Airport",
        "Charlotte Douglas International Airport",
        "Guangzhou Baiyun International Airport",
        "Chengdu Shuangliu International Airport",
        "Shenzhen Bao'an International Airport",
        "Chongqing Jiangbei International Airport"
      )
    )
  ) |>
  ggplot(aes(x = year, y = passengers, color = airport, group = airport)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_continuous(labels = scales::comma) +
  labs(
    title = "Top 6 Busiest Airports by Year",
    subtitle = "Total passengers for airports appearing in the yearly top 6, 2020-2025",
    x = "Year",
    y = "Passengers",
    color = "Airport"
  ) +
  theme_minimal() +
  theme(
    legend.position = "bottom",
    legend.text = element_text(size = 6),
    legend.title = element_text(size = 8)
  ) +
  guides(color = guide_legend(ncol = 2))
# Monte Carlo Simulation Function ----
# Generates n random (x, y) points within a specified rectangle
mcSimulation <- function(n, xMin, xMax, yMin, yMax) {
  data.frame(
    x = runif(n, min = xMin, max = xMax),
    y = runif(n, min = yMin, max = yMax)
  )
}
# Set seed for reproducibility ----
set.seed(184)

# Rectangle bounds ----
```

```r
xMin <- -4
xMax <- 4
yMin <- 0
yMax <- 0.4
rectArea <- (xMax - xMin) * (yMax - yMin)

# Helper function to build a single MC plot for a given n ----
makeMCPlot <- function(n) {
  simData <- mcSimulation(n, xMin, xMax, yMin, yMax)

  mcResults <- simData |>
    mutate(
      curveY = dnorm(x),
      flag = if_else(y <= curveY, "on/below", "above")
    )

  estIntegral <- rectArea * mean(mcResults$flag == "on/below")

  ggplot(mcResults, aes(x = x, y = y, color = flag)) +
    geom_point(alpha = 0.7, size = 1.2) +
    stat_function(fun = dnorm, linewidth = 1, color = "navy") +
    scale_color_manual(values = c("on/below" = "steelblue", "above" = "coral")) +
    coord_cartesian(xlim = c(xMin, xMax), ylim = c(yMin, yMax)) +
    labs(
      title = paste0("Monte Carlo Integration, n = ", format(n, big.mark = ",")),
      subtitle = paste("Estimated Integral =", round(estIntegral, 4)),
      x = "x",
      y = "y",
      color = "flag"
    ) +
    theme_minimal()
}

# Build the four plots at different resolutions ----
plot10    <- makeMCPlot(10)
plot100   <- makeMCPlot(100)
plot1000  <- makeMCPlot(1000)
plot10000 <- makeMCPlot(10000)

# Combine into small multiple with {patchwork} ----
(plot10 + plot100) / (plot1000 + plot10000) +
  plot_layout(guides = "collect") &
  theme(legend.position = "bottom")
```